

smallgameagent 实验报告

基于 LLM/VLM 与规则的小游戏 Agent

smallgameagent 项目组

2026 年 7 月 21 日

目录

1	smallgameagent 实验报告（第三轮：分层架构 + 批量框架）	3
1.1	0. 一页结论	3
1.2	1. 分层多 Agent 架构（实验 F）	3
1.2.1	设计	3
1.2.2	实现	4
1.2.3	批量实验结果	4
1.3	2. Node.js 高级逻辑移植（实验 H）	4
1.4	3. 批量实验框架（实验 G）	5
1.4.1	架构	5
1.4.2	使用方式	5
1.4.3	产出	5
1.4.4	轨迹数据格式	5
1.5	4. 同学系统对比分析	6
1.6	5. 多游戏泛化实验（第四轮）	6
1.6.1	5.1 Generic fallback 让 22 游戏可驱动	6
1.6.2	5.2 Probe 终止假阳性修复	6
1.6.3	5.3 修正后多游戏结果（5 游戏 × 2 模式）	7
1.6.4	5.4 全游戏自动校准（auto_calibrate.py -all, 22 游戏）	7
1.6.5	5.5 Tap-to-Move 驱动策略	7
1.7	6. 全游戏 × 多模式批量矩阵	8
1.7.1	6.1 A_full（7 个 joystick 游戏 × 4 模式 × 2 seeds = 48 runs）	8
1.7.2	6.2 B_tap（15 个 tap-only 游戏 × 2 模式 × 2 seeds = 60 runs）	8
1.8	7. 云端 API 策略生成 vs 纯 rule	9
1.9	8. VLM 视觉管线	9
1.10	9. 训练数据生成	9
1.11	10. L2 输出契约修复	9

1.12	11. 后续建议	10
1.13	12. 多 Provider 云端 API 配置	10
1.14	13. 规则在线更新架构（保守方案 A）	11
1.14.1	13.1 三层结构	11
1.14.2	13.2 触发条件	11
1.14.3	13.3 更新产物 schema	11
1.14.4	13.4 实现	12
1.15	14. 本地 VLM 基准实验（RTX 5060 Laptop 8 GB）	12
1.16	15. Agent 通信扩展	12
1.17	16. 规则在线更新 A/B 消融实验	12
1.18	17. 训练数据增量	13

1 smallgameagent 实验报告（第三轮：分层架构 + 批量框架）

> 2026-07-20。配套：__CODE__0000__（方案）、__CODE__0001__（过程数据）。
> 本轮重点：分层多 Agent 架构、Node.js 高级逻辑移植、批量实验框架、数据采集管线。

1.1 0. 一页结论

- __BOLD__0000__：实现 L0 规则（每步 ~0ms）+ L1 本地 VLM（每 5 步 ~5s）+ L2 云端 API（每 15 步 ~3s）三层决策。批量实验中 hierarchical 模式 composite=0.150，但 L1 因本地 VLM 未启动而退化为 L0+L2；L2 kimi-k2.7-code 的思考链输出导致 JSON 解析失败。__BOLD__0001__：架构可行，但需要 (a) 本地 VLM 常驻 + (b) 更强的 L2 输出契约。
- __BOLD__0004__：__CODE__0000__ + __CODE__0001__ 支持多游戏 × 多模式 × 多 seed 矩阵实验，自动采集逐步轨迹 JSONL。8 runs 产出 __CODE__0002__ + 8 个轨迹文件 + __CODE__0003__。
- __BOLD__0001__：soft target lock（防 target thrashing）、guide-signature change detection（检测 guide 路径变化）、coin demand override（强制导航到 coin table）已移植到 __CODE__0000__。但 soft lock 在 tap-guide 场景下反而降低了 tap 频率（rule composite 从 0.150 降到 0.10），已修复：tap 后释放 lock。
- __BOLD__0000__：批量实验中 multi-bus 和 multi-bus-memory 均达 __BOLD__0001__（2 seed 一致），确认记忆读回 + 总线通信的组合是当前最佳配置。
- __BOLD__0002__：__CODE__0000__ 已接入 __CODE__0001__，每步写入 JSONL（player/action/keyNumbers/reason），可直接用于后续 VLM 微调。
- __BOLD__0000__：674 passed, 0 failed, ruff 全绿。

1.2 1. 分层多 Agent 架构（实验 F）

1.2.1 设计

层	模型	频率	延迟	职责
L0 执行层	Rule engine (tap-guide)	每步	~0ms	跟随当前 plan 执行 move/tap
L1 战术层	本地 VLM (gemma-4-E4B)	每 5 步或 stuck	~5s	看截图判断当前状态，修正短期目
L2 战略层	云端 API (kimi-k2.7-code)	每 15 步或 phase 切换	~3s	看 probe state 做长程规划

1.2.2 实现

- `__CODE_0000__`： `__CODE_0001__` 类， `__CODE_0002__` 按频率调度三层。
- `__CODE_0000__`：注册 `__CODE_0001__` 模式。
- L2 输出 `__CODE_0000__`，存入 `__CODE_0001__`。
- L1 输出 `__CODE_0000__` 或 `__CODE_0001__`，覆盖 L0 动作。

1.2.3 批量实验结果

模式	Mean Composite	Mean Activity	Mean Latency	L1 calls	L2 calls
<code>__BOLD_0000__</code>	<code>__BOLD_0000__</code>	1.000	21.9s	—	—
multi-bus-memory	0.215	0.931	23.4s	—	—
hierarchical	0.150	0.000	205.1s	6	2
rule	0.101	0.672	34.7s	—	—

`__BOLD_0000__`：

1. hierarchical 的 activity=0 是因为 L1（本地 VLM）未启动，L2 的 JSON 被 kimi-k2.7-code 的思考链截断，导致 L0 rule engine 收到的 macro_plan 为空，退化为纯 wait。
2. L2 调用 2 次（step 0 和 step 15），每次 ~3s；L1 调用 6 次（step 0/5/10/15/20/25），每次因连接拒绝而 ~0s 但记录 warning。
3. `__BOLD_0000__`：(a) 本地 VLM 常驻服务；(b) L2 用”只输出 JSON”系统提示或 tool calling；(c) L1 失败时 fallback 到 PIL 视觉分析。

1.3 2. Node.js 高级逻辑移植（实验 H）

从同学的 `__CODE_0000__`（2185 行）移植 3 个机制：

机制	作用	实现
Soft target lock	选定 target 后锁定 8 步，防 target thrashing	<code>__CODE_0000__</code> ：a
Guide-signature change	检测 guide 路径/角度变化，触发重规划	<code>__CODE_0000__</code> ：p
Coin demand override	station 需要 coins 但玩家没有时，强制导航到 coin table	<code>__CODE_0000__</code> ：松

`__BOLD_0001__`：soft lock 在 tap-guide 场景下导致 tap 后仍锁定旧 target，新 target 无法被选中，tap 频率从 24/30 降到 19/30。`__BOLD_0002__`：tap 动作后立即释放 lock（`__CODE_0000__`）。

1.4 3. 批量实验框架 (实验 G)

1.4.1 架构

```
““
src/experiments/batch_runner.py —BatchConfig + run_batch()
src/experiments/analyze_batch.py —读取 batch_results.json → Markdown 表格
src/experiments/exp_batch_matrix.py —预定义矩阵配置 (1 game × 4 modes × 2 seeds)
““
```

1.4.2 使用方式

```
““python
from src.experiments.batch_runner import BatchConfig, run_batch

config = BatchConfig(
games={"SSD_00461P01": "/path/to/game.html"},
modes=["rule", "multi-bus", "multi-bus-memory", "hierarchical"],
seeds=[42, 123],
max_steps=30,
collect_dataset=True, # 自动写入轨迹 JSONL
output_dir="batch_results",
)
results = asyncio.run(run_batch(config))
““
```

1.4.3 产出

- __CODE__0000__: 汇总所有 run 的 composite/activity/details
- __CODE__0000__: 每步 player/action/keyNumbers/reason
- __CODE__0000__: Markdown 对比表格

1.4.4 轨迹数据格式

每行一个 JSON 对象:

```
““json
{"player": {"x": 2.53, "z": 12.78}, "action": "tap", "keyNumbers": {"_failCount": 0},
"reason": "tap_guide_dist=1.95"}
““
```

““

可直接用于：

1. VLM 微调数据（next_probe_action / probe_action_effect 任务）
2. 离线回放分析（stall 检测、target thrashing 诊断）
3. A/B 对比可视化

1.5 4. 同学系统对比分析

对比 __CODE_0000__ 的 Node.js 系统：

维度	同学 Node.js	我们 Python
游戏覆盖	12 游戏完整驱动	1 游戏 profile + 22 游戏 HTML 无
控制循环	coin override + soft lock + guide signature + A* routing	soft lock + guide sig + coin override
每步延迟	~2.9s (Playwright + probe)	~0.8s (rule) / ~22s (multi-bus)
数据采集	__CODE_0000__ 写 JSONL	__CODE_0000__ 写 JSONL (本
VLM 训练	10,336 样本 7 任务 QLoRA	15,083 样本 7 任务 (processed-runs
多 Agent	无	6 角色总线 + 记忆 + Critic

1.6 5. 多游戏泛化实验（第四轮）

1.6.1 5.1 Generic fallback 让 22 游戏可驱动

__CODE_0000__ 新增 __CODE_0001__ (floating joystick + 单位基线，未校准) + __CODE_0002__。__CODE_0003__ 对无 profile 游戏不再抛错，改用 generic 驱动（移动方向不可靠但 tap 坐标有效）。

1.6.2 5.2 Probe 终止假阳性修复

__BOLD_0002__: Cocos 引擎标志 __CODE_0000__ (含“finish”)被 probe WIN 正则误命中，以及 Logo/火堆等常驻 UI 被标“completion-like”——导致 00482/00342 在第一个动作后被误报 __CODE_0001__，1 步假阳性、composite 虚高 0.700。

__BOLD_0002__: __CODE_0000__ 改为佐证制——要求 win/victory 面板节点 / 胜利 analytics / 非 __CODE_0001__ 管理器的强胜利标志之一，排除 lose/fail (保留 00461 失败重试)。

游戏	校准?	模式	步数	composite	activity	tap	stall
00461 塔防	cal	rule	25	0.106	0.71	17	7
00461 塔防	cal	multi-bus-memory	25	___BOLD_0000___	1.00	24	0
00482 砍树	GEN	rule	25	0.150	0.00	0	24
00482 砍树	GEN	multi-bus-memory	25	0.150	0.00	0	24
00736 养蛙捕鱼	GEN	rule	25	0.269	0.79	20	5
00736 养蛙捕鱼	GEN	multi-bus-memory	25	___BOLD_0000___	1.00	25	0
00342 建造合并	GEN	rule	25	0.150	0.00	0	24
00342 建造合并	GEN	multi-bus-memory	25	0.150	0.00	0	24
00532 瀑布巨木	GEN	rule	25	0.150	0.00	0	24
00532 瀑布巨木	GEN	multi-bus-memory	25	0.150	0.00	0	24

1.6.3 5.3 修正后多游戏结果（5 游戏 × 2 模式）

___BOLD_0000___:

1. ___BOLD_0000___——与已校准 00461 持平。记忆读回补偿了未校准基线：记住成功 tap 模式后 activity 0.79→1.00、stall 5→0。
2. ___BOLD_0000___ 在所有 5 游戏上一致成立。
3. 00482/00342/00532 的 activity=0 是 ___BOLD_0000___：未校准基线 → 方向全错 → 需该游戏的 profile 校准。
4. 10 个轨迹 JSONL 已采集 (___CODE_0000___)，可直接用于 VLM 微调。

1.6.4 5.4 全游戏自动校准 (auto_calibrate.py -all, 22 游戏)

对全部 22 个 ___CODE_0000___ 游戏跑自动校准（4 方向 joystick 脉冲 + 返回脉冲 + warmup + 重试 + moveByCocosInput 回退），得到 joystick 基线或识别为非 joystick 游戏。

___BOLD_0000___:

类型	数量	游戏
A 类 (joystick 驱动)	5	00440 清障通车、00483 吸沙抽水、00496 电网抓丧尸、00517 末世旅店、00526 末世旅店
A 类 (已有校准)	2	00461 塔防、00736 捕鱼
B 类 (tap-to-move)	15	00219、00332、00342、00382、00394、00427、00434、00475、00482、00526、00532、00533、00534、00535、00536、00537
C 类 (probe 失败)	0	—

___BOLD_0000___:

1.6.5 5.5 Tap-to-Move 驱动策略

为 B 类 (tap-to-move / 自动移动) 游戏新增 ___CODE_0000___ 驱动：不走路，直接 tap guide 目标的屏幕坐标 (probe 的 design→CSS 映射，无需 joystick 基线校准)。

游戏	screen_right	screen_down
00440 清障通车	(-6.42, 2.34)	(-2.34, -6.42)
00483 吸沙抽水	(3.41, -3.41)	(2.42, 2.42)
00496 电网抓丧尸	(0.75, -0.90)	(0.82, 3.57)
00517 末世旅店	(7.42, -0.00)	(0.00, 9.44)
00522 地下炸矿	(4.92, 0.00)	(-0.27, 3.46)

15 个 B 类游戏已自动填充 __CODE_0001__ profile。

__CODE_0000__ 函数自动分类：A (joystick) / B (tap-only) / C (probe 失败),
__CODE_0001__ 自动选择驱动类型。

1.7 6. 全游戏 × 多模式批量矩阵

1.7.1 6.1 A_full (7 个 joystick 游戏 × 4 模式 × 2 seeds = 48 runs)

游戏	rule	multi-bus-memory	multi-bus	hierarchical
00440 清障通车	0.184	0.156	0.156	0.150
00461 塔防	0.113	0.300	0.297	0.150
00483 吸沙抽水	0.139	0.300	0.300	0.150
00496 电网抓丧尸	0.275	0.150	0.150	0.150
00517 末世旅店	0.150	0.150	0.150	0.150
00522 地下炸矿	0.215	0.240	0.300	—

- __BOLD_0000__ (activity=1.00, stall=0)。
- __BOLD_0000__——确定性策略比记忆启发式更有效。
- __BOLD_0000__ (activity=0)：云端 API 的 L2 macro-plan 没有被 L0 规则引擎有效执行。

1.7.2 6.2 B_tap (15 个 tap-only 游戏 × 2 模式 × 2 seeds = 60 runs)

结果	游戏	数量
__BOLD_0000__	00382、00394、00475、00526、00532、00594、00669、00742	__BOLD_0000__
0.150	00219、00332、00342、00427、00434、00482、00733	7

__BOLD_0000__——tap-only 驱动有效：tap guide 目标的屏幕坐标产生真实交互。7/15 为 0.150 (tap 有效但无移动, activity=0)。

游戏	rule	multi-bus-memory	hierarchical (API)
00461 塔防	0.106	0.056	0.150
00736 捕鱼	0.275	0.237	0.150

1.8 7. 云端 API 策略生成 vs 纯 rule

云端 API 的 L2 macro-plan 没有被 L0 规则引擎有效执行 (activity=0)。L2 输出需要更强的行动契约。

1.9 8. VLM 视觉管线

游戏	probe_only	pil_vision	vlm_gemma
00461 塔防	0.044	___BOLD_0000___	0.150
00736 捕鱼	0.269	0.269	0.150

- ___BOLD_0000___ (0.044→0.087, tap 7→14, stall 17→10)。
- ___BOLD_0000___ (activity=0, tap=0) ——本地小模型输出太慢且无法解析为有效动作。

1.10 9. 训练数据生成

从 125 条批量实验轨迹 (7 joystick + 15 tap-only 游戏 × 4 模式 × 2 seeds) 离线转换为同事 7 任务训练格式：

任务	新增样本	合并后总计
probe_action_effect	+3040	5531
information_gain_judgment	+3040	6094
progression_grounding	+3165	5806
pulse_response_grounding	+159	1594
failure_recovery	+9	23
___BOLD_0000___	___BOLD_0000___	___BOLD_0000___

转换器：___CODE_0000___，纯离线计算（相邻步 diff → changed_fields / information_gain / displacement / stall diagnosis）。

1.11 10. L2 输出契约修复

___BOLD_0000___：云端 API 的 L2 macro-plan 是抽象文本，L0 无法执行 → hierarchical 全部 activity=0。

___BOLD_0001___: L2 prompt 改为要求输出可执行指令列表 (tap/move 带坐标), 存入 ___CODE_0000___ 动作队列, L0 逐步弹出执行。

___BOLD_0000___:

模式	composite	L2 calls	latency/step
hierarchical (v3, 指令队列)	0.055	1	18.1s
rule_baseline	0.114	0	0.84s
multi_bus_memory	0.134	0	0.88s

___BOLD_0000___: L2 指令队列架构正确 (24/25 步来自 L2 指令), 但 ___BOLD_0001___——没有视觉, 坐标全是幻觉。composite 反而更差 (0.055 vs rule 0.114)。

___BOLD_0001___: L2 改为输出目标名称 (___CODE_0000___), L0 用 probe 的 screenPosition 自行映射坐标。这样 L2 不需要视觉, L0 保留几何准确性。

1.12 11. 后续建议

1. ___BOLD_0001___: 用 probe 的 ___CODE_0000___ 脉冲自动测量每游戏的 screen→world 基线, 批量生成 profile。
2. ___BOLD_0000___: systemd 保持 gemma-4-E4B 运行, 让 hierarchical L1 可用。
3. ___BOLD_0000___: kimi-k2.7-code 加”只输出 JSON”系统提示。
4. *___ITALIC_0000___ routing 移植 **: 从 00864/00867 驱动移植。

1.13 12. 多 Provider 云端 API 配置

为支持同学在实验中使用不同云端 LLM/VLM, 我们扩展了 ___CODE_0000___:

- 新增 ___CODE_0000___, 统一接入 OpenCodeGo、Kimi、DeepSeek、MiMo(provider 名 ___CODE_0001___)、Qwen;
- 通过 ___CODE_0000___ 文件配置各 provider 的 ___CODE_0001___ / ___CODE_0002___ / 默认模型, ___CODE_0003___ 已加入 ___CODE_0004___, 绝不会被提交;
- 切换 provider 只需设置 ___CODE_0000___ 环境变量, 模型名随 provider 自动默认;
- 保留 ___CODE_0000___ 完全兼容, 现有调用无需修改;
- Kimi 系列自动省略 ___CODE_0000___ 参数, 避免 Console Go 代理返回 400。

Provider	默认文本模型	默认视觉模型	base_url
opencodego	deepseek-v4-flash	mimo-v2.5	__CODE_0000__
kimi	kimi-k2.7-code	kimi-k2.6	__CODE_0000__
deepseek	deepseek-chat	deepseek-chat	__CODE_0000__
xiaomi	mimo-v2.5	mimo-v2.5	__CODE_0000__
qwen	qwen-coder-plus	qwen-vl-plus	__CODE_0000__

新增 __CODE_0000__ 单测覆盖 provider 路由、env 读取、参数覆盖、认证文件 fallback。

1.14 13. 规则在线更新架构（保守方案 A）

针对同学提出的「规则作为底层，需要修改时让上两层更新」的需求，我们设计并实现了在线规则更新机制：

1.14.1 13.1 三层结构

- __BOLD_0000__：零延迟执行，读取内存参数；
- __BOLD_0000__：看截图输出结构化视觉上下文，辅助云端 API 理解画面；
- __BOLD_0000__：根据触发条件和视觉上下文，输出结构化规则更新 JSON。

1.14.2 13.2 触发条件

- __CODE_0000__ 连续 N 步低于阈值（默认 0.15）；
- __CODE_0000__ 停滞计数超过阈值（默认 5 步）；
- L0 与 L2 决策冲突持续 K 步；
- 世界模型检测到空间/时间一致性违例（__CODE_0000__ stale 传播）。

1.14.3 13.3 更新产物 schema

```
“‘json
{
  "update_type": "param|memory_entry|phase_contract",
  "target": "rules.upgrade_threshold",
  "reason": " 金币未攒够就升级导致往返",
  "payload": {"upgrade_threshold": 500},
  "confidence": 0.82
}
```

““

1.14.4 13.4 实现

- 新增 __CODE_0000__ : __CODE_0001__、__CODE_0002__、__CODE_0003__、__CODE_0004__；
- 扩展 __CODE_0000__：在 __CODE_0001__ 中检查触发条件，满足时调用 L2 请求规则更新，解析并应用；
- 规则更新只修改内存参数和 __CODE_0000__，不修改源码文件（保守方案 A）；
- 新增 __CODE_0000__ 14 个单测。

1.15 14. 本地 VLM 基准实验 (RTX 5060 Laptop 8 GB)

使用 LM Studio 的 CUDA12 llama.cpp 后端，4-bit 量化 + q4_0 KV-cache，在 __CODE_0000__ 上跑结构化视觉提取：

模型	解析成功	平均延迟	生成 tok/s	说明
gemma-4-E4B-it-Q4_K_M	3/3	4.42 s	58.8	本地最佳，输出可直接解析
Qwen3.5-9B-Q4_K_M	1/3	14.67 s	47.9	仅一帧成功，其余输出被 markdown fence
Qwen3.5-4B-Q4_K_M	0/3	~10.6 s	~67	速度够快但格式控制不稳

结论：__BOLD_0000__，延迟已接近在线逐步控制的可接受范围；Qwen 系列需要更强的”no markdown fences” prompt 约束。

1.16 15. Agent 通信扩展

- 在 __CODE_0000__ 的 __CODE_0001__ 中新增 __CODE_0002__；
- 为后续多 Agent 协作中「规则更新提议 → Critic 评估 → MemoryCurator 落地」的流水线预留消息类型；
- 相关单测通过。

1.17 16. 规则在线更新 A/B 消融实验

在 00461 塔防上对比 rule / multi-bus-memory / hierarchical（规则更新触发开启）：

模式	composite	activity	move	tap	stall	墙钟
rule	0.112	0.750	6	18	6	29.6 s
multi-bus-memory	0.038	0.250	18	6	18	32.1 s
hierarchical（规则更新开启）	0.150	0.000	0	0	24	367.9 s

___BOLD_0000___:

1. ___BOLD_0000___: L2 输出被 kimi 思考链截断, L1 本地 VLM 未启动, 导致 activity=0 (全部 wait)。但 consistency 得分高, 所以 composite 反而比 rule 高——这是「不动作就不会错」的假象。
2. ___BOLD_0000___: 本次使用独立内存文件, 无历史记录, 表现反而不如 rule。
3. ___BOLD_0000___: 当 stall streak 达到阈值时确实调用了 L2, 但 L2 返回的是抽象 planning JSON 而非规则更新 JSON, 说明需要把 planning 和 rule-update 的 prompt 彻底分离。

___BOLD_0000___: 为 hierarchical 模式禁用默认 L2 planning, 只保留规则更新触发; 或者让 L2 planning 输出目标名称而非坐标。

1.18 17. 训练数据增量

新增 rule_update_ab_results 轨迹后, 运行 ___CODE_0000___ 生成新的 7 任务样本:

任务	本次新增	合并后总计
probe_action_effect	+3,112	8,643
information_gain_judgment	+3,112	9,206
progression_grounding	+3,240	9,046
pulse_response_grounding	+181	1,775
failure_recovery	+9	32
___BOLD_0000___	___BOLD_0000___	___BOLD_0000___

数据覆盖 rule / multi-bus-memory / hierarchical 三种模式, 可用于后续 QLoRA 微调。

5. ___BOLD_0001___: CI/CD 中跑 ___CODE_0000___, 每次代码变更自动对比 composite。